

GAME ECONOMY & TOKENOMICS · MASTER DOCUMENT

Run a loop. Own the ground. Get paid when people cross it.

The complete design — how the game works, how the token works, and exactly how to build the contracts. Written so anyone can follow it.

Prepared for	Version	Covers	Status
Team & partners	1.0 · Master	Game · Tokenomics · Contracts	Ready to build

START HERE

How to read this

Four parts. The first explains the ideas from scratch. The last is a technical specification for whoever writes the contracts. The middle two are the design itself.

Part A · The basics

What a token is, why these economies fail, and what "owning land" means here.

Part B · The game

How you take ground, how you keep it, how attacks work, and how crossing someone's land pays them.

Part C · The tokenomics

Where tokens come from, where they go, why supply falls, and what the numbers look like.

Part D · How to build it

Contract-by-contract specification: state, functions, events, roles, tests, migration.

A1	Two systems, not one	The game's money and the company's money
A2	What a token actually is	From absolute zero
A3	Why these economies usually fail	Four patterns, and our answer to each
A4	One token, two states	Locked and liquid explained
A5	What owning land means	And what it deliberately does not mean
B6	The core loop	Run a loop, claim the inside
B7	Taking ground	Free land versus minted land
B8	The seven-day siege	How a deed changes hands
B9	The toll	2% from everyone who crosses
B10	Land tiers	What upgrading buys, and what it does not
B11	Minting	The day your earnings become real money
C12	Supply	The cap, the schedule, the reason it cannot run away
C13	How tokens are created	The effort curve
C14	Where tokens go	Every sink, priced and explained
C15	Staking, sponsors and voting	The rights layer
C16	The complete picture	Everything on one page
C17	Does it balance?	The numbers, and where they break
C18	What stops it being gamed	Every attack and its answer
D19	Architecture	Three layers, and what each is responsible for
D20	The contract map	Seven contracts and how they connect
D21–24	Contract specifications	State, functions, events, invariants
D25	Oracle and signatures	Exact payloads and verification
D26	Security, roles, testing, migration	Plus the mistakes not to repeat
E	Glossary and summary	Every term in one line

A NOTE ON THE NUMBERS

Every figure here is a **modelled estimate**, not a forecast. There are no players and no revenue yet. The structure has been checked arithmetically; the participation rates and prices are assumptions, and several will turn out to be wrong. Nothing here is financial advice or a promise of returns.

Two systems, not one

Nearly every misunderstanding about this kind of product comes from mixing up two separate things. Get this right and the rest is easy.

SYSTEM ONE

The game's money

- Players earn tokens by moving
- They spend them inside the game
- Spending destroys them
- Almost no cash enters or leaves

The game's fuel — not our income.

SYSTEM TWO

The company's money

- Sponsors pay to reach walkers
- Subscriptions for tools and privacy
- Events and city programmes
- A small fee on land trades

Ordinary money, ordinary invoices.

The token makes the game work. The business pays the bills. They meet in only two places.

The company's revenue does not depend on the token being worth anything.

A café paying to sponsor the street outside its door writes an ordinary invoice. Someone subscribing for route analytics pays an ordinary subscription. Neither cares what the token trades at. That separation is deliberate, and it is the difference between a business and a bet.

WHY IT MATTERS SO MUCH

The failed products in this category had only System One. Their entire business was the token, so when the token fell they had no income and nothing to fall back on. **MovenRun is built so that if the token were worth nothing at all, the company would still have paying customers and the game would still be playable.**

Where the two systems touch

CROSSING POINT	WHAT HAPPENS
Sponsors funding prizes	A sponsor may put existing tokens into a prize pot. This moves tokens that already exist — it never creates new ones.
Players topping up	A player short of tokens for something they want can buy a small amount. The purchase routes straight into the action and is destroyed.

That is the entire overlap. Everywhere else the two systems run independently.

What a token actually is

Forget everything you have heard about crypto for two minutes. The underlying idea is far simpler than the noise around it.

Start with a notebook nobody owns

Imagine a notebook in a village square. Anyone can write in it, everyone can read it, and once something is written it can never be erased — only added to. Thousands of people keep identical copies, so if one person forges their copy, every other copy disagrees and the forgery is rejected.

That is a **blockchain**. It is not a currency or a company. It is a way of keeping a record that no single party controls.

A token is a number written in that notebook

If the notebook can record who owns what, it can record quantities too. A token is a balance: *"this account holds 350 units."* That is the whole idea.

What makes tokens interesting is not the technology — it is the **rules** attached to them, which are also written into the notebook and cannot be quietly changed afterwards. How many can ever exist. Who may create new ones. What they can be used for. What destroys them. **Those rules are what people mean by "tokenomics".**

The rules are the product

A cap that cannot be raised. A mint that only fires on proof of movement.
A burn that fires on every purchase.
All enforced by code, not by promises.

Nobody can cheat them

Not even us. We cannot print extra, raise the cap, or reverse a burn once the contract is deployed.

That is the only reason to use it

A company database can be edited overnight. A public ledger cannot. When we say a player genuinely owns their land, this is what makes it true.

Two kinds of token, and why we use both

KIND	WHAT IT IS	OURS
Interchangeable	Every unit is identical, like banknotes. Balances, not objects.	\$MOVE — the currency you earn by moving and spend on everything
Unique	Each one is distinct and individually identifiable, like a title deed to a specific house.	Deeds — one per piece of ground, carrying its history and its owner

The unique kind became famous through cartoon profile pictures, which is an unhelpful association. The underlying idea is just *"this specific thing belongs to this specific person, recorded where nobody can revoke it."* Land registries are the natural use, and ours is literally a land registry.

THE SENTENCE TO REMEMBER

A token is a balance in a ledger nobody owns, governed by rules nobody can secretly change.
Everything else in this document is about what those rules should say.

Why these economies usually fail

Move-and-earn products have been tried before and most collapsed. Four patterns explain nearly all of it, and every major decision in this design is an answer to one of them.

1 Supply grew with the player count

The usual design mints a fixed number of tokens per player per day. Twice the players means twice the tokens. Growth — the thing every product wants — becomes the thing that destroys it.

Our answer: the pool is a fixed schedule. Twice the players means the same tokens, divided further. Growth can never inflate supply. *Section C12.*

2 The main way to spend created another earner

In those products the biggest purchase was a new item or creature that then earned tokens of its own. So the largest "sink" was really an **investment in more inflation**. Every burn today bought more minting tomorrow, and the books looked balanced right up until they did not.

Our answer: nothing you can buy produces tokens. Gear wears out, charge drains, fortifications expire, upgrades raise your *share* of a fixed pot rather than the pot itself. *Section C14.*

3 Spending stopped but minting did not

Players kept earning no matter what, but they only bought things while they felt confident. When sentiment turned, purchases collapsed and minting carried on unchanged — so supply grew fastest exactly when it should have shrunk.

Our answer: most of what we mint is **locked** and can only be spent inside the game. Locked tokens get burned regardless of what anyone thinks the price is. Our burn does not depend on confidence. *Section A4.*

4 Free accounts were token farms

Anyone could sign up, earn, sell, and repeat with a new account. The economy funded its own attackers.

Our answer: free accounts earn only locked tokens, which have no market value and never convert. There is nothing to farm. *Section A4.*

THE DESIGN RULE THAT FOLLOWS FROM ALL FOUR

Decide how tokens leave before deciding how they enter. That order is unusual and it is why this economy has reasons to exist beyond new buyers arriving.

One token, two states

This is the most important mechanism in the design. Everyone earns the same token; whether it can be sold depends on how it was obtained.

Newly minted tokens are locked.
Tokens that already existed move freely.
One line that decides every case in the whole economy.

STATE ONE

Locked \$MOVE

Earned by moving. Most of the supply.

✓ Buys everything in the game

✓ Can be staked for a liquid yield

✗ Cannot be sent, sold or converted

✗ Never unlocks, even later

STATE TWO

Liquid \$MOVE

Tokens that already existed, changing hands.

✓ Everything locked can do

✓ Transferable and sellable

✓ Carries voting and staking rights

✓ Buys and sells deeds on the market

Same token, same ledger, same supply cap. Only the transfer permission differs.

Which is which — the complete list

HOW YOU GOT IT	STATE	WHY
Reward for a walk, run or ride	Locked	Newly minted
Reward from a contest you won	Locked	Newly minted
Toll paid by someone crossing your land	Liquid	Transferred from another player
Yield on your staked balance	Liquid	Paid from the toll pool
Sponsor challenge prize	Liquid	Sponsor deposited existing tokens
Monthly prize draw	Liquid	Funded externally
Earnings from land <i>after</i> you mint it	Liquid	Minting switches the tap
Bought on the market	Liquid	Already existed

WHY THIS IS THE WHOLE GAME, ECONOMICALLY

Locked tokens can only be spent in-game, and everything spent in-game is destroyed. So **every locked token ever minted eventually leaves the supply for good**. Free players are not a drain on the economy — they are the engine that shrinks it. And because a free account can never produce anything sellable, mass signups are safe rather than dangerous.

What owning land means here

The phrase sounds grand. The mechanism is closer to a parking permit than to buying a field — and being precise about that protects everyone.

The world is divided into permanent cells

We lay a honeycomb over the entire planet using an open standard called H3. Every point on Earth belongs to exactly one cell with a permanent identifier, about **0.74 km²** each — roughly a small neighbourhood. Two people standing on the same corner are always in the same cell, anywhere in the world.

When you claim ground, the record says: *"cell #8828308281 belongs to account #4821."* That record is your land. For the game's purposes, **the record is the land**.

WHAT IT IS NOT — SAY THIS CLEARLY TO ANYONE WHO ASKS

A deed is **a game asset, not real-world property**. It gives no rights over the physical ground, no access, no planning permission, and no relationship with whoever actually owns the land. Anyone implying otherwise is misrepresenting the product.

Why the scarcity here is real rather than invented

There is only one of each place

The cell containing your city's main square exists exactly once, for ever. No amount of demand creates a second one. That is genuine scarcity, not a number a team chose.

It must be earned by being there

Ground is claimed by physically moving through it. Capital cannot substitute for presence, so a fund cannot buy a city before players arrive.

Busy ground is worth more

Land earns from people crossing it. A quiet cul-de-sac earns almost nothing however much you spend on it; a high street earns constantly.

Holding it costs effort

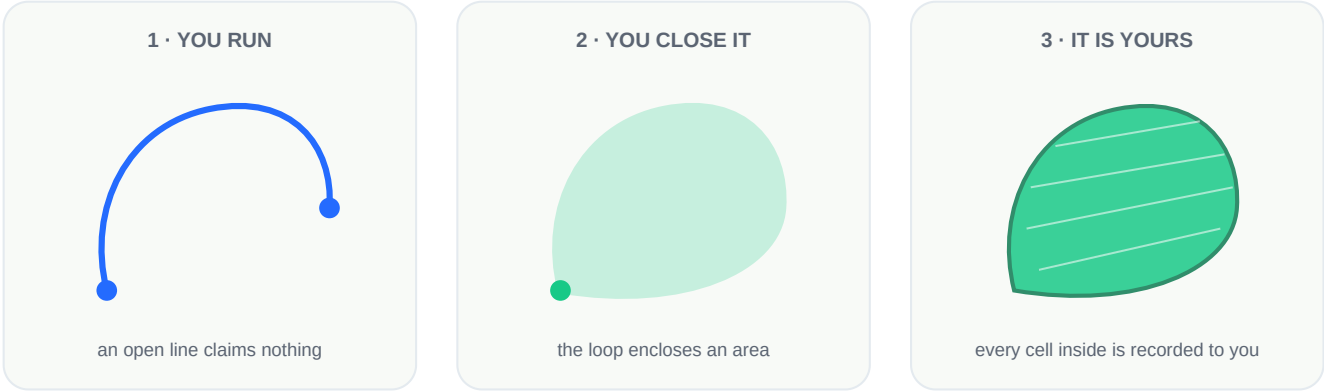
Rivals can take it, and land you never visit decays and eventually returns to the map. Land is a commitment, not a collectible.

WHY THIS IS HARD TO COPY

A competitor can clone the app in a quarter. They cannot clone the fact that the people in your neighbourhood already hold the ground around your street, or the sponsor relationships in that city. **The moat is geographic and social, and it only exists where we have already won.**

The core loop

Run a closed loop and everything inside it becomes yours. Not step-by-step tile collecting — enclosure. Bigger and smarter routes take more map.



The route is the boundary. Everything it surrounds becomes your ground.

Why enclosure instead of stepping

- **It rewards planning, not pace.** A clever three-kilometre loop takes more ground than a fast ten-kilometre straight line. Ordinary walkers compete with athletes on equal terms.
- **It makes routes interesting.** You start thinking about where to turn, not how fast to go — which is better for health and much better as a game.
- **It suits cities.** Blocks, parks and rivers naturally form loops, so the map reads like the place you actually live.

The rules, precisely

RULE	DETAIL
Closing the loop	Your finish must come within about 100 m of your start. Below that it stays an open route and claims nothing.
What you claim	Every H3 cell whose centre falls inside the enclosed area, plus the cells the route itself passes through.
Maximum area	Capped per session so one enormous loop cannot swallow a city. Beyond the cap, only the boundary cells count.
Overlaps	If your loop encloses ground someone else holds, the normal taking rules apply — see the next section.
Verification first	Nothing is claimed until the route passes plausibility checks. A rejected route claims nothing and earns nothing.

THE PLANNING TOOL

Before you go out you can draw a route and see what it would take: how much new ground, whose land you would cross, what it would cost you in tolls and what it might earn. **This turns a walk into a decision**, which is most of what makes a territory game enjoyable.

Taking ground

Two completely different rules, depending on whether the owner has minted their land. This difference is what gives minting its value.

UNMINTED GROUND

Run through it. It's yours.

- No cost, no waiting, no permission
- One session is enough — a single player can take a district in a morning
- You lose it exactly the same way
- Defend it by running it again yourself

Fast, fluid and free. Most of the map.

MINTED GROUND — A DEED

You must win a siege.

- Crossing it pays the owner a toll
- Taking it needs a declared, staked, seven-day contest
- The owner can fortify, up to three times
- Winner takes it; the loser's stake burns

Slow, costly and decisive. Real property.

Minting is the moment ground stops being a scoreboard and becomes something worth defending.

Side by side

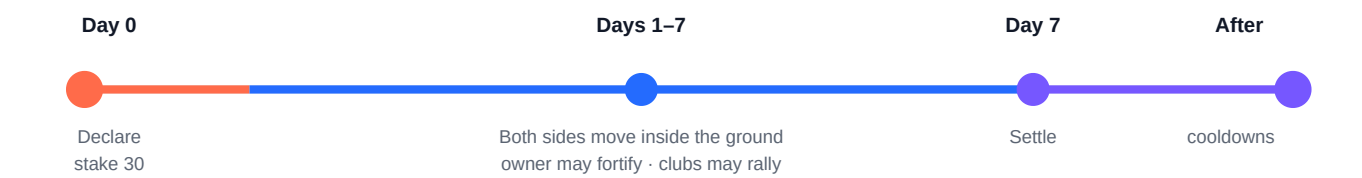
	UNMINTED	MINTED DEED
How it changes hands	Someone runs through it	Only by winning a seven-day siege
Cost to take	Nothing	A staked declaration, burned if you lose
Crossers pay you	Nothing	2% of what they earned there
Your earnings from it	Locked	Liquid, from mint day onward
Can be sold	No	Yes, on the marketplace
Can be sponsored	No	Yes
If you stop playing	Taken almost immediately	Decays slowly, then reopens after a long absence

WHY BOTH LAYERS EXIST

The free layer keeps the game **fast and exciting** — ground changes hands constantly, newcomers can take real territory on their first run, and nothing feels locked up by people who arrived early. The minted layer gives committed players something that **stays theirs** and pays them. Neither works alone: all-free means nothing is ever really yours, all-minted means newcomers find a closed map.

The seven-day siege

How a minted deed changes hands. Long enough that it cannot be sniped, short enough to stay exciting, and decided by movement rather than money.



STEP	COST	WHAT HAPPENS
Declare	30 tokens	You name the deed. The owner is notified immediately. A seven-day window opens. Your stake is held.
Compete	free	Both sides record movement <i>inside that ground</i> . Counted once per person per day, so it rewards turning up rather than distance.
Fortify	28 each, max 3	The owner may add +20% to their score, up to three times. Burned whether they win or lose.
Rally	free	Club mates who move through the ground add to the side they back. One contribution per person per day.
Loyalty	—	Long-held ground defends better: +25% after 90 days, +50% after 180, +75% after a year.
Settle	—	Higher score wins. The deed transfers automatically — the loser cannot refuse or block it . The loser's stake is burned.
Cool off	—	A beaten attacker cannot return to that deed for 30 days. A defender who just survived gets 14 days of peace.

Can one person beat a club?

YES — AND THIS IS DELIBERATE

Because scoring counts **distinct people per day**, a club of thirty who each turn up once scores thirty. A single determined runner who turns up every day for seven days scores seven days of full effort plus loyalty and fortification — which beats them. **Nobody is locked out for having no friends, and nobody wins by buying anything:** fortification helps at the margin but cannot close a real gap in movement.

The escrow matters technically: the deed is held by the contract from the moment a siege opens, so settlement cannot be blocked by an uncooperative loser. This was a genuine defect in our first contracts and it must not be repeated — see Part D.

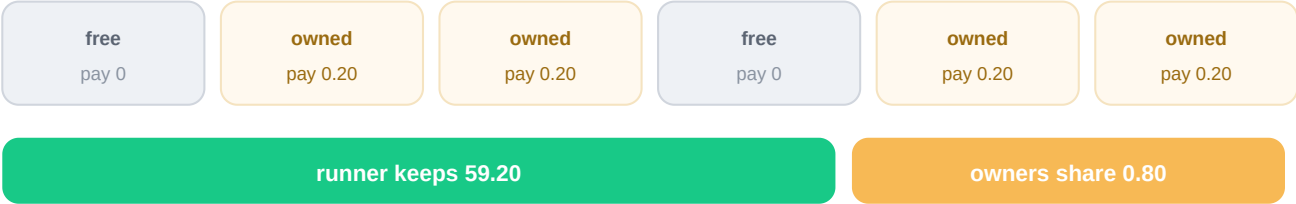
The toll

Cross someone's minted land and 2% of what you earned *there* goes to them. Cross ten people's land and you still pay about 2% in total — it cannot pile up.

2% of what you earned *inside each cell* — not 2% of your whole session.

The toll is bounded by how far you went, never by how many owners you happened to pass.

One 60-token session crossing six cells, four of them owned



Total toll: 1.3% of the session. Nothing is minted — the toll is a transfer between players.

YOU CROSS...	YOU PAY	WHY
6 cells, none owned	0%	Most of the map is unowned
6 cells, 4 owned	1.3%	Only the owned portions are charged
10 cells, all owned	2.0%	The practical ceiling
20 cells, all owned	2.0%	Still 2% — each cell is a smaller slice of the same session
Hard cap, whatever happens	5%	So a bad route can never feel like a punishment

Your own land is always free to cross. Unminted ground charges nothing — another reason minting is worth paying for.

THREE PROBLEMS SOLVED BY ONE MECHANIC

Location starts to matter — a deed on a busy path is worth far more than one in a dead end, so choosing ground becomes a real decision rather than a land grab. **Owners are paid without printing anything** — every token collected came from a runner, not from new supply. And **owners now want more runners nearby**, which is exactly the incentive a growing network needs.

Land tiers

Upgrading raises your share of every toll and your defence. It never creates income on its own — **empty land earns nothing however much you spent on it.**

TIER	UPGRADE	TOLL SHARE	DAILY CEILING	WHAT ELSE
1	—	1.00×	40	Base ground
2	140	1.25×	100	A quest slot, custom colour
3	380	1.55×	220	+10% defence, banner
4	900	1.90×	440	+20% defence, sponsor-ready
5	2,000	2.30×	820	+30% defence, your name on the map

The same tier on different streets

WHERE IT IS	TIER 1 EARNS	TIER 3 EARNS	TIER 5 EARNS
Dead suburb — 2 crossers a day	0.40	0.62	0.92
Quiet street — 12 a day	2.40	3.72	5.52
Busy path — 45 a day	9.00	13.95	20.70
High street — 140 a day	28.00	43.40	64.40

A tier-1 plot on a high street beats a tier-5 plot in a dead suburb by about thirty times. Spending cannot substitute for choosing good ground and drawing people to it. That is what keeps this a territory game rather than a yield farm — and it is why we never quote a payback period for an upgrade.

The account cap

No account earns more than 900 tokens a day from land, ever.

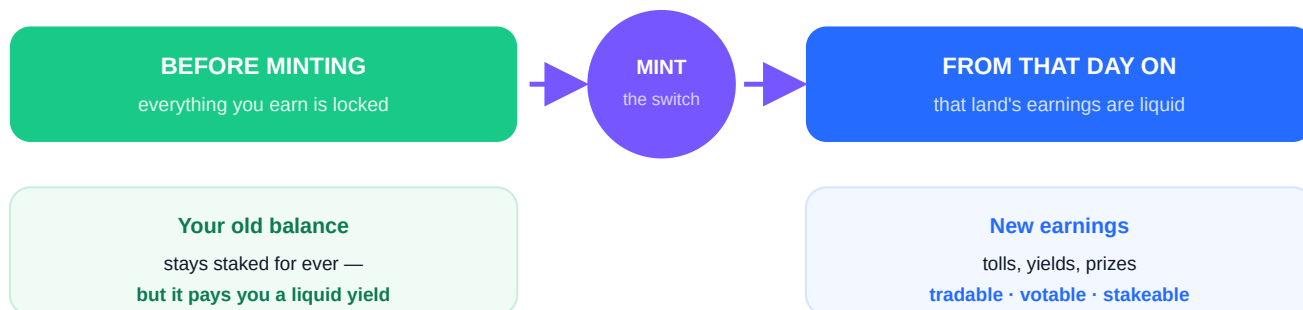
However many plots you hold, however high the tier.

HOLDINGS	RAW CEILING	ACTUALLY PAID
1 tier-5 plot	820	820
3 tier-5 plots	2,460	900
20 tier-5 plots	16,400	900

Past roughly one well-placed tier-5 plot, more land buys status, defence and district control — but no more income. A whale who corners a city collects no more than a committed local, **and every token they spent upgrading was burned.**

Minting — the day earnings become real

One decision changes everything about a piece of land. It is also the single clearest reason for a player to spend money.



Nothing already earned is retroactively unlocked. That is what stops anyone farming free accounts and cashing out later.

What minting costs and what it gives

	BEFORE MINTING	AFTER MINTING
Cost	Free	400 tokens, or the cash equivalent
Someone runs through it	They take it	They pay you a toll
To take it from you	One run	A seven-day siege they must win
Your earnings	Locked	Liquid from that day
Can you sell it	No	Yes
Can it be sponsored	No	Yes
Voting rights	No	Yes, through the tokens it pays you

Two guards on minting

- **You must have held the ground for a period first.** Minting is an upgrade to land you already hold, never a way to buy land you have never visited.
- **The ground must have real crossing traffic.** Cells nobody passes through cannot be minted, so nobody can lock up a whole empty city cheaply and wait.

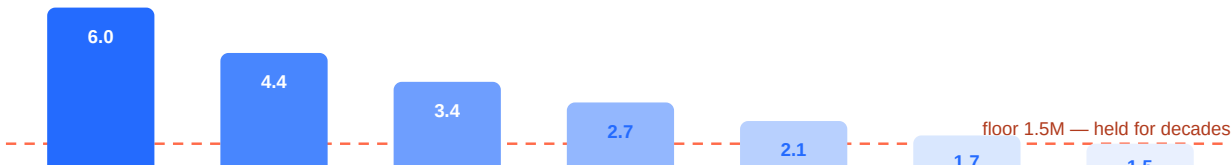
THE RISK WE ARE WATCHING

If minting is cheap and the good paths are obvious, experienced players could mint the best ground before newcomers understand the game. The two guards above are our answer, but **this is the parameter most likely to need changing after the first city.** It should launch conservatively and loosen if nothing bad happens.

Supply — the cap and the schedule

A hard limit on how many tokens can ever exist, and a release schedule that does not care how many people play.

RULE	SETTING
Maximum supply — fixed in the contract, raisable by nobody	1,000,000,000 \$MOVE
Reserved for player rewards	300,000,000 (30%)
Released per season (90 days)	6M at launch, -10% each season, floor 1.5M — about 43 years



Why a fixed schedule is the single most important rule

The failed products minted per player. Ours mints per season, whatever the headcount — so more players always means a smaller share each, never more tokens:

IF THE GAME DOUBLES IN SIZE	THEIR DESIGN	OURS
Tokens created	Doubles	Unchanged
Per-player reward	Unchanged	Halves
Tokens destroyed by spending	Roughly doubles	Roughly doubles
Net effect on supply	Inflation grows	Deflation grows

Growth is therefore the safest thing that can happen to us. Every new player adds spending without adding a single token to the schedule. In the products that failed, growth was what eventually killed them.

The rest of the supply

ALLOCATION	SHARE	RELEASE
Player rewards	30%	The season schedule above
Ecosystem & sponsors	15%	Milestone tranches over at least five years
Team	12%	18-month cliff, 48-month vest, public wallets
Investors	8%	12-month cliff, then 36-month linear vest
Treasury	8%	Timelocked budgets, quarterly disclosure
Liquidity	5%	Locked or under a disclosed mandate
Security & bug bounty	2%	Incident or milestone based
Unissued reserve	20%	Stays unissued unless governance releases it

How tokens are created

Only verified movement creates tokens. And the curve deliberately pays for consistency rather than distance.



PART OF THE SESSION	COUNTS AT	PLUS
First 1 km	100%	+0.5 Effort for your first session of the day — turning up beats going far.
1 – 3 km	60%	
3 – 6 km	30%	Capped at 4.0 Effort per person per day, so nobody can grind past the ceiling.
6 – 10 km	10%	
Beyond 10 km	0%	Walking, running and cycling all count the same. There is no minimum speed.

What that pays four different people

PLAYER	EFFORT / MONTH	TOKENS / SEASON
Short walker — 750 m, 5 days a week	27	~36
Casual — 2 km, 5 days a week	46	~60
Regular — 4 km, 6 days a week	78	~103
Dedicated — 8 km, 7 days a week	115	~152

THREE THINGS THIS CURVE BUYS AT ONCE

Inclusion — a short daily walk is a genuinely competitive way to play, so this is a health product rather than a race. **Anti-farming** — driving, cycling for hours or looping a route all day earns almost nothing extra, so the cheapest cheat is also the least profitable. **Habit** — the daily bonus rewards turning up, which is what actually changes people's lives.

Where tokens go

Six things to spend on. Every one recurs, every one is something players want, and every token spent is destroyed for good.

1 · Gear

Craft boots, jackets and packs from materials found in different terrain — parks give one thing, hills another, night walks another. Looks and capability, never extra earnings.

2 · Recharge

Gear and land hold a charge that drains with use. Walking your own ground recharges it free; paying recharges it instantly. The most frequent purchase in the game.

3 · Upgrade

Land levels up: a bigger share of every toll, stronger defence, more quest slots and style options. Never more income by itself.

4 · Attack

Declaring a siege on someone's deed costs a stake. Refunded if you win, burned if you lose. The cost of ambition.

5 · Defend

Fortifying during a siege, up to three times. Burned whether you win or lose. The cost of keeping what you have.

6 · Style

Colours, map themes, banners, titles, share cards. Pure sink, no advantage, and the easiest thing to sell to someone enjoying themselves.

SPEND	COST	WHO DOES IT	PER PLAYER	WHY THAT PRICE
Recharge	15	80%	12.0	Small and frequent — free if you visit instead
Upgrade to tier 2	140	8%	11.2	The first real goal worth saving for
Upgrade to tier 3	380	3%	11.4	A serious commitment
Upgrade to tier 4	900	1%	9.0	For people who have chosen good ground
Upgrade to tier 5	2,000	0.25%	5.0	Rare, prestigious, name on the map
Mint a deed	400	2%	8.0	The gateway to liquid earnings
Attack	30	15%	4.5	Low enough to try, real enough to think about
Fortify	28	18%	5.0	Painful but affordable, up to three times
Style	12	40%	4.8	Guilt-free and repeatable
Total spending			70.9	against about 60 earned

THE CALIBRATION THAT MATTERS

A typical player earns about **60** tokens a season and finds about **71** worth of things they want. The eleven-token gap is filled by collecting tolls from land they own, or by buying a small amount — roughly **25 pence a month** at a realistic price. Small enough that nobody feels pushed, large enough that across a city it becomes the demand that absorbs what sellers sell.

Staking, sponsors and voting

Three ways to hold real, sellable value — all three move tokens that already exist rather than minting new ones.

Staking — your locked balance can still work

A locked balance is not dead money. You can **stake** it, and stakers share the tolls collected across the network. Your principal stays locked for ever; **what it pays you is liquid and yours to sell**.

	DETAIL
What you stake	Any part of your locked balance
What you get	A share of the toll pool, in proportion to your stake, paid liquid
Where it comes from	Runners crossing minted land — existing tokens, never new ones

Stake is committed for one season and released at the boundary. This is deliberately not a savings account: no promised rate, and it can never pay out more than the network actually collected. If nobody is running, nobody is paid.

Sponsors — outside money entering the game

HOW IT WORKS	DETAIL
Who	Local businesses, gyms, race organisers, brands
What they buy	Verified footfall past their door — distinct visitors, repeat rate, month-on-month lift
What they deposit	Cash for a service fee, or existing tokens into a prize pot — never newly minted ones
What players get	Bonus rewards for passing through — and those prizes are liquid, even for free players
The protection	A sponsor can add rewards, never take any away, gate anything, or pick a winner

Voting — who decides the settings

Holders of liquid tokens vote on the parameters of this economy — toll rate, sink prices, season rules, treasury spending. Any holder above a small threshold may propose. Voting is snapshot-based, so nobody can buy a vote at the last minute, and timelocked, so every change is visible before it lands.

CONTROL	RULE
Voting weight	From a snapshot taken when the proposal is created
Delay before execution	Two days minimum, published in advance
What cannot be voted	The supply cap; the rule that locked tokens never convert

WHAT A FREE PLAYER CAN ALREADY SELL, FROM DAY ONE

Staking yield, sponsor prizes and monthly draws are **all liquid**, because all three move tokens that already existed. A free player who never spends a penny can still earn sellable value. What nobody can do is mint sellable tokens out of thin air — that is the whole protection.

The complete picture

Everything in Parts A to C on one page. Green is locked, blue is liquid, red is destroyed, gold is real money.



Most of what is minted is locked, and everything locked is eventually burned. Tolls, yields and prizes move existing tokens only.

Does it balance?

Illustrative models of the structure, not forecasts of adoption. Figures are per 90-day season — and note that the released column never changes.

SCENARIO	PLAYERS	RELEASED	DESTROYED	CAN BE SOLD	BURN : SELL
Pilot city	5,000	6.0M	4.8M	0.99M	4.8×
Base case	100,000	6.0M	4.8M	0.99M	4.8×
Strong growth	500,000	6.0M	5.2M	0.66M	7.8×
Demand shock	100,000	6.0M	4.4M	1.32M	3.3×

How the free/paid mix changes everything

SHARE OF PLAYERS WHO ARE FREE	DESTROYED	CAN BE SOLD	BURNED PER TOKEN SOLD
60%	4.36M	1.32M	3.3×
70%	4.77M	0.99M	4.8×
80%	5.18M	0.66M	7.8×

Read that table twice: the more free players there are, the more deflationary the economy becomes. In the products that failed, growth meant more tokens hitting the market. Here it means more tokens locked into a form that can only be burned.

The shock case. In a downturn people stop minting deeds, so more of the pool goes out liquid: burn falls, selling rises, the ratio drops from 4.8× to 3.3×. It gets worse — and it stays positive, because locked tokens keep burning whatever anyone thinks the price is. **Our burn does not depend on confidence.**

What would actually break this

RISK	WHAT HAPPENS, AND WHAT WE DO
Players hoard instead of spending	The biggest risk by far. If locked balances sit unspent, burn stalls. Wear, charge drain and seasonal deadlines push against it — but if they feel like a tax rather than part of the game, players quit instead of paying. Wear must launch generously.
Nobody buys the top-up	Sinks shrink and the economy runs smaller. Survivable, but it removes the demand that absorbs selling — and with fewer deeds minted, tolls, yield and governance all stay thin.
Deed sniping	Experienced players lock up the best ground early. Guarded by the holding period and traffic minimum — the likeliest parameter to need changing.

THE HONEST SUMMARY

The structure is sound and fails safely: supply is capped, the pool never grows with the player base, nothing you can buy produces tokens, and most of every release can only be burned. **The calibration, though, is entirely ours** — about eight numbers here are estimates, and the first city exists to replace them with measurements.

What stops it being gamed

Written as an attacker would write it. If a defence here is weak, better that we find it than that somebody else does.

THE ATTACK	WHY IT DOES NOT PAY
Fake the GPS	Speed, accuracy, continuity and duration checks. Each route is usable once. Nothing is created without a signature from the verification service.
Drive the route	Anything implying more than about 80 km/h is rejected — and beyond 10 km the reward is zero anyway.
Run marathons to farm	The effort curve flattens at 10 km and is capped at 4.0 per person per day.
Mass free accounts	Free accounts earn only locked tokens, which have no market value and never convert. There is nothing to farm.
Farm free, mint later, cash out	Minting never unlocks old earnings. Everything earned before the mint stays staked for ever.
Lap my own land for tolls	You never pay a toll on your own ground, so there is nothing to collect from yourself.
Two accounts paying each other tolls	A toll is 2% of an already-capped, already-verified reward. Round-tripping loses 98% each way — it is pure loss.
Buy every plot on a high street	The 900-a-day account cap. Past about one good plot, more land adds no income at all.
Club of thirty takes every deed	Sieges count distinct people per day with sharply diminishing returns, and clubs face a citywide deed limit.
Endless attacks to grief one owner	A beaten attacker is locked out of that deed for 30 days, and each attempt costs a stake that burns.
One phone, many accounts	Device attestation, one home claim per verified person, and cells count distinct movers with device clusters counting once.
Wash-trade deeds to fake a market	A marketplace fee each way, loyalty resets to zero on transfer, and the ground still has to be defended.

THE PRINCIPLE UNDERNEATH ALL TWELVE

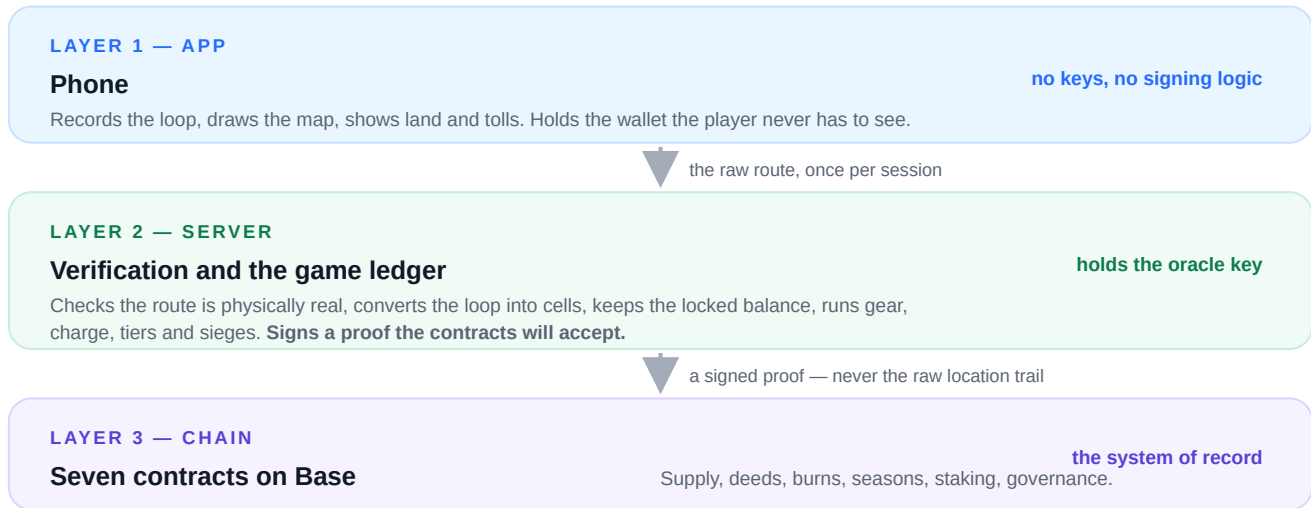
Anything that can be automated earns nothing sellable, and anything sellable requires being in a real place over real time. Every defence above is an application of that one rule — and it is also why the free tier can be completely open. An account that costs nothing to make is worth nothing to farm.

STATED HONESTLY

No location system is perfectly spoof-proof and we do not claim otherwise. A determined attacker with real devices moving in real places can extract some value — but they would, in effect, be playing the game. The goal is not perfection; it is that **cheating costs more than playing**.

Architecture

Three layers. Getting the boundary right is what keeps fees near zero and the economy honest.



What lives where, and why it matters

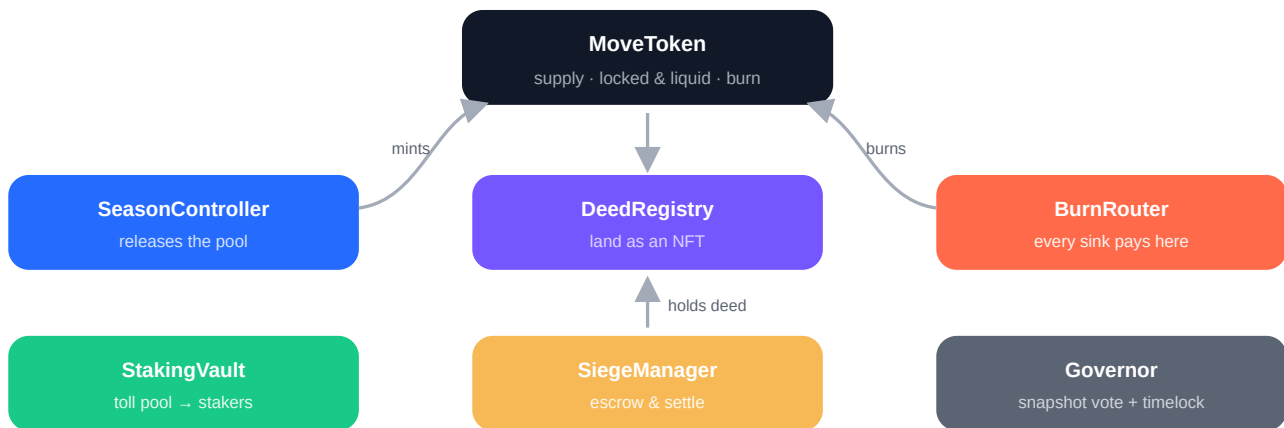
THING	WHERE	REASON
Locked balance	Server	Changes constantly. On-chain it would cost a fortune in fees for no benefit — it can never be transferred anyway.
Unminted land, tiers, charge	Server	Same reason. Ground changes hands many times a day.
Gear, collections, sieges	Server	Game state. Only the burn that pays for them touches the chain.
Liquid balance	Chain	Must be transferable and independently verifiable.
Minted deeds	Chain	The whole point is that we cannot revoke them.
Every burn	Chain	A burn only counts if anyone can check it.
Season pool, staking, votes	Chain	The rules must not be editable by us.

THE BOUNDARY RULE

If it can be transferred or sold, it lives on the chain. If it cannot, it lives on the server. That single line decides every design question about where to put something, keeps ordinary play free of fees, and means a busy player might touch the chain only a handful of times a season.

The contract map

Seven contracts. Each does one job, and only one contract may ever create tokens.



CONTRACT	STANDARD	ONE-LINE JOB
MoveToken	ERC-20 + AccessControl	Holds the supply cap and both balance states. The only contract that can mint.
SeasonController	AccessControl	Releases the fixed pool each season and splits it locked/liquid.
BurnRouter	AccessControl	Single entry point for every sink. Destroys tokens and emits a public receipt.
DeedRegistry	ERC-721	Minted land. <code>tokenId</code> is the H3 cell id. Holds tier and mint date.
SiegeManager	AccessControl + ReentrancyGuard	Escrows the deed, takes stakes, settles the winner. Cannot be blocked by the loser.
StakingVault	AccessControl + ReentrancyGuard	Accepts locked stake, distributes the toll pool as liquid yield.
Governor	Snapshot + Timelock	Parameter changes, treasury spending, with a mandatory delay.

THE INVARIANT TO ENFORCE ABOVE ALL OTHERS

Only SeasonController may call `mint`, and only within the season schedule. No gameplay path, no upgrade, no deed, no siege and no staking action may create a token. If a future feature needs to mint, the feature is wrong.

Contract one — MoveToken

The token itself. It holds the cap, it holds both balance states, and it is the only place in the whole system where a token can be created or destroyed.

What makes it unusual

A normal ERC-20 stores one number per address. This one stores **two**: a locked balance and a liquid balance. Locked tokens can be spent inside the game but cannot be transferred to another wallet or sold. Liquid tokens behave exactly like any normal token.

Everything else — the cap, the accounting, the events — works the way an ordinary ERC-20 works, so wallets, explorers and exchanges see nothing strange.

THE DESIGN RULE THAT KEEPS THIS SIMPLE

`balanceOf(a)` returns **liquid only**. That is what makes the contract compatible with every wallet and exchange on the chain without a single special case. The locked balance is exposed separately through `lockedBalanceOf(a)`, and the sum through `totalBalanceOf(a)`.

State

```
// ----- supply -----
uint256 public constant MAX_SUPPLY = 1_000_000_000 ether;
uint256 public totalMinted;      // only ever increases
uint256 public totalBurned;      // only ever increases

// ----- balances -----
mapping(address => uint256) private _liquid;  // transferable
mapping(address => uint256) private _locked;  // spendable in-game only
uint256 public totalLiquid;
uint256 public totalLocked;

// ----- roles -----
bytes32 public constant MINTER_ROLE = keccak256("MINTER_ROLE"); // SeasonController ONLY
bytes32 public constant SPENDER_ROLE = keccak256("SPENDER_ROLE"); // BurnRouter, StakingVault,
SiegeManager
```

THE ONE INVARIANT THAT MUST NEVER BREAK

`totalMinted - totalBurned == totalLiquid + totalLocked`, and `totalMinted <= MAX_SUPPLY`. Assert both in every test. If a change makes either fail, the change is wrong — no exceptions, no migrations, no admin override.

Functions

```
// --- creation (SeasonController only) -----
function mintLocked(address to, uint256 amount) external onlyRole(MINTER_ROLE);
function mintLiquid(address to, uint256 amount) external onlyRole(MINTER_ROLE);
// both revert if totalMinted + amount > MAX_SUPPLY

// --- spending (game contracts only) -----
function spendFrom(address from, uint256 amount)
    external onlyRole(SPENDER_ROLE) returns (uint256 lockedUsed, uint256 liquidUsed);
// spends LOCKED FIRST, then liquid. Moves nothing — the caller decides
// whether the amount is burned or credited elsewhere.
```

```

function burnFrom(address from, uint256 amount) external onlyRole(SPENDER_ROLE);
function convertLockedToLiquid(address who, uint256 amount) external onlyRole(SPENDER_ROLE);
//   used in exactly two places: sponsor prizes, and staking yield.

// --- reads -----
function balanceOf(address a) public view returns (uint256);      // liquid only
function lockedBalanceOf(address a) external view returns (uint256);
function totalBalanceOf(address a) external view returns (uint256);
function totalSupply() public view returns (uint256);             // liquid + locked
function circulatingSupply() external view returns (uint256);     // liquid only

```

Why "locked first" matters

Every spend inside the game consumes locked tokens before it touches liquid ones. This single ordering rule does most of the economic work in the document:

- Free players — whose balance is entirely locked — automatically pay for everything with tokens that can never be sold. **Their spending removes supply without ever adding sell pressure.**
- Paying players naturally hold on to their liquid tokens, because the game keeps quietly spending their locked ones first.
- Nobody has to choose, understand, or manage two currencies. There is one balance in the app, one number, one token.

Events

```

event Minted(address indexed to, uint256 amount, bool locked, uint256 season);
event Spent(address indexed from, uint256 lockedUsed, uint256 liquidUsed, bytes32 indexed reason);
event Burned(address indexed from, uint256 amount, bytes32 indexed reason);
event Unlocked(address indexed who, uint256 amount, bytes32 indexed reason);

```

Every event carries a `reason` code (`"TIER_UP"` , `"SIEGE"` , `"REPAIR"` , ...). This is what makes the public dashboard possible: anyone can add up burns by reason directly from the chain, with no trust in us required.

Contract two — DeedRegistry

Minted land, as an NFT. One deed per map cell, forever. This is the contract that turns a place on a map into something a person actually owns.

The identifier trick

The token id is the map cell id. The map already gives every cell in the world a unique 64-bit number; we use that number directly as the NFT id. There is no lookup table, no registry of which id means which place, and no way for two deeds to point at the same ground.

```
// The H3 cell id, cast straight to uint256. Nothing else is ever used.
uint256 tokenId = uint256(cellId);

// Uniqueness is therefore free: ERC-721 already forbids minting an id twice.
```

WHAT THIS ELIMINATES

Two players cannot claim overlapping ground, because "overlapping" is not representable. A cell is claimed or it is not. This removes an entire class of disputes, and an entire class of bugs, at the cost of one design decision.

State

```
struct Deed {
    uint8    tier;           // 1..5
    uint32   mintedAt;       // day index – earnings switch on-chain from here
    uint32   lastTierUp;     // cooldown anchor
    uint16   integrity;      // 0..10000 (basis points) – falls with traffic
    uint32   colour;         // owner-chosen map colour, RGB
}

mapping(uint256 => Deed) public deeds;           // tokenId => deed data
mapping(address => uint256) public deedCount;    // for the per-account cap
uint256 public maxDeedsPerAccount;              // governance parameter
uint8    public constant MAX_TIER = 5;
```

Functions

```
// --- creating a deed -----
function mintDeed(uint256 cellId, MintProof calldata proof) external;
// 1. verify the oracle signature (see §25)
// 2. require deedCount[msg.sender] < maxDeedsPerAccount
// 3. MoveToken.spendFrom(msg.sender, price) -> BurnRouter.burn(...)
// 4. _safeMint(msg.sender, cellId); deeds[cellId] = Deed(1, today, ...)

// --- upgrading -----
function tierUp(uint256 cellId) external;
// requires ownership, tier < MAX_TIER, cooldown elapsed, integrity == full
// burns the tier price; increments tier; sets lastTierUp = today

function repair(uint256 cellId, uint16 points) external;
// burns points * repairPrice; integrity += points (capped at 10000)

// --- administration by the owner -----
function setColour(uint256 cellId, uint32 rgb) external;
function tokenURI(uint256 cellId) public view returns (string memory);
```

```
// --- called only by SiegeManager -----  
function forceTransfer(uint256 cellId, address to) external onlyRole(SIEGE_ROLE);  
function damage(uint256 cellId, uint16 points) external onlyRole(TRAFFIC_ROLE);
```

Two rules that stop land becoming a money printer

Tiers pay on traffic, not on time

A tier-5 deed on a dead street earns nothing. Upgrading multiplies what crossings pay you; it does not add a per-day drip. A deed that nobody walks across is worth exactly what it cost, which is the correct answer.

Integrity falls as traffic rises

Every crossing lowers integrity slightly. The busiest, most profitable land is the land that needs repairing most often — so the best-earning deeds are also the biggest token sinks. Success is self-taxing.

THE CAP THAT HAS TO EXIST

`maxDeedsPerAccount` is not optional. Without it a single well-funded account buys a whole city, every toll flows to one wallet, and the map stops being a game. With it, land ownership spreads by construction. Start it low; governance can raise it once real distribution data exists.

Events

```
event DeedMinted(uint256 indexed cellId, address indexed owner, uint256 pricePaid);  
event TierChanged(uint256 indexed cellId, uint8 from, uint8 to, uint256 burned);  
event IntegrityChanged(uint256 indexed cellId, uint16 from, uint16 to);  
event ColourSet(uint256 indexed cellId, uint32 rgb);
```

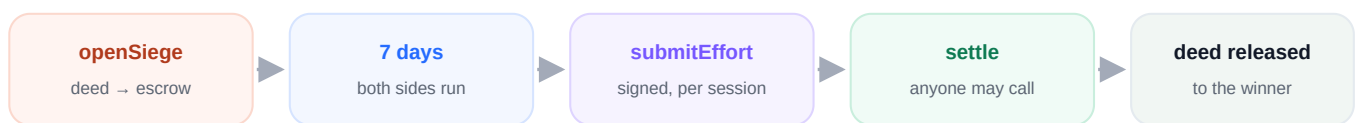
Contract three — SiegeManager

Attacks, defence, and the transfer of a deed from one owner to another. This is the hardest contract to get right, because it is the only one where somebody loses.

THE MISTAKE THIS CONTRACT EXISTS TO PREVENT

In a naive design the deed stays in the defender's wallet during the siege, and the winner has to ask the contract to move it at the end. A defender who is losing can simply sell or transfer the deed before settlement, and the attacker's stake is gone for nothing. **The fix is escrow: the deed leaves the defender's wallet the moment a siege opens, and neither side can touch it until the contract settles.**

Lifecycle



Once opened, nothing can stop settlement. The deed is not in anybody's wallet until it ends.

State

```

struct Siege {
    address attacker;
    address defender;
    uint256 cellId;
    uint64 opensAt;
    uint64 closesAt;      // opensAt + 7 days
    uint256 attackStake;   // escrowed, spent locked-first
    uint256 defenceStake;
    uint256 attackEffort;  // accumulated from signed sessions
    uint256 defenceEffort;
    bool settled;
}

mapping(uint256 => Siege) public sieges;           // cellId => live siege
mapping(uint256 => uint64) public immuneUntil;    // cooldown after a siege ends
uint64 public constant SIEGE_DURATION = 7 days;
uint64 public constant IMMUNITY = 14 days;
uint16 public defenderAdvantageBps;              // e.g. 11500 = defender effort x1.15
  
```

Functions

```

function openSiege(uint256 cellId, uint256 stake) external nonReentrant;
// require block.timestamp > immuneUntil[cellId]
// require sieges[cellId].opensAt == 0           (one at a time)
// require stake >= minStakeFor(tier)
// MoveToken.spendFrom(attacker, stake) -> held by this contract
// DeedRegistry.forceTransfer(cellId, address(this)) <-- ESCROW

function reinforce(uint256 cellId, uint256 stake) external nonReentrant;
// defender (or an ally) adds stake at any time before closesAt

function submitEffort(uint256 cellId, EffortProof calldata p) external;
  
```

```
// oracle-signed; adds to attackEffort or defenceEffort
// each proof carries a nonce; replaying one reverts

function settle(uint256 cellId) external nonReentrant;
// callable by ANYONE once block.timestamp >= closesAt
// score = effort * stakeWeight; defence score * defenderAdvantageBps / 10000
// winner takes the deed; loser's stake is split by BurnRouter;
// immuneUntil[cellId] = block.timestamp + IMMUNITY
```

THREE PROPERTIES TO TEST EXPLICITLY

1. `settle` must be callable by any address, so a losing defender cannot stall by simply never calling it. 2. Settling twice must revert. 3. If the defender never shows up at all, the attacker wins on effort alone — a solo attacker against an absent owner must be able to take the land. That last one is the whole point of the mechanic.

Where the loser's stake goes

SHARE	DESTINATION	WHY
70%	Burned	Every siege permanently shrinks supply. Conflict is the largest single sink in the game.
20%	Winner	Enough to make attacking worthwhile, not enough to make it free.
10%	Season pool	Recycled into next season's rewards rather than into anyone's pocket.

The winner's own stake is returned in full. Only the loser's stake is split — so defending successfully costs nothing but the effort, and attacking badly costs real tokens.

Contract four — SeasonController

The only contract allowed to create tokens. It releases a fixed pool each season, pays out sessions, and takes the toll on the way through.

The schedule is code, not policy

Each season's pool is written into the contract as a decaying series. Nobody — not us, not governance, not an emergency — can mint more than the schedule allows in a given season. This is what makes "the supply cannot run away" a fact about the code rather than a promise in a document.

```
uint64 public constant SEASON_LENGTH = 90 days;
uint256 public constant SEASON_0_POOL = 6_000_000 ether;
uint16 public constant DECAY_BPS = 1_000; // -10% per season
uint256 public constant POOL_FLOOR = 1_500_000 ether; // never falls below

function poolFor(uint256 season) public pure returns (uint256 p) {
    p = SEASON_0_POOL;
    for (uint256 i = 0; i < season; ++i) p = p * (10_000 - DECAY_BPS) / 10_000;
    if (p < POOL_FLOOR) p = POOL_FLOOR;
}
```

READ THAT LOOP ONCE MORE

Season 0 releases 6 million tokens. Season 1 releases 5.4 million. Season 2, 4.86 million. It keeps shrinking until it reaches 1.5 million, and then it stays there. Against a 300-million player reserve, that schedule runs for roughly **43 years** before the reserve is exhausted — and every burn along the way pushes that date further out.

Paying a session — and taking the toll

```
function claimSession(SessionProof calldata p) external nonReentrant {
    _verify(p); // $25 - oracle signature + nonce
    require(seasonSpent[currentSeason] + p.gross <= poolFor(currentSeason), "POOL");
    require(dailyPaid[p.runner][p.day] + p.gross <= DAILY_CAP, "CAP");

    uint256 tollTotal;
    for (uint256 i = 0; i < p.crossings.length; ++i) { // bounded: max 40
        uint256 cut = p.gross * TOLL_BPS / 10_000; // TOLL_BPS = 200 → 2%
        cut = cut * tierMultiplier(p.crossings[i]) / 100;
        tollTotal += cut;
    }
    if (tollTotal > p.gross * TOLL_MAX_BPS / 10_000) // TOLL_MAX_BPS = 500 → 5%
        tollTotal = p.gross * TOLL_MAX_BPS / 10_000; // hard ceiling, always

    _payLandowners(p.crossings, tollTotal);
    token.mintLocked(p.runner, p.gross - tollTotal); // or mintLiquid if minted land
}
```

THE CEILING IS NOT DECORATION

Without `TOLL_MAX_BPS`, a route crossing forty owned cells would take eighty per cent of the runner's reward and the game would be unplayable in exactly the cities where it is most successful. **The cap makes the toll a small, predictable cost no matter how built-up the map becomes.** Crossing twenty owned cells still costs about the same as crossing three.

Contract five — [BurnRouter](#)

Every sink in the game routes through one contract. Nothing anywhere else is permitted to call `burnFrom` directly.

```
function route(address from, uint256 amount, bytes32 reason)
    external onlyRole(SINK_ROLE) nonReentrant
{
    (uint256 lockedUsed, uint256 liquidUsed) = token.spendFrom(from, amount);
    uint256 toBurn      = amount * BURN_BPS    / 10_000;    // 7 000 → 70%
    uint256 toStakers   = amount * STAKER_BPS  / 10_000;    // 2 000 → 20%
    uint256 toSeason    = amount - toBurn - toStakers;      // remainder → 10%

    token.burnFrom(address(this), toBurn);
    vault.fund(toStakers);
    seasonController.recycle(toSeason);
    emit Routed(from, amount, reason, lockedUsed, liquidUsed, toBurn);
}
```

Two reasons this is a separate contract rather than a helper function. First, one place to audit: if you want to know every way a token can be destroyed, you read this file and no other. Second, one place to change: if governance later decides burns should be 75% instead of 70%, that is one parameter in one contract, not a change spread across six.

Contract six — [StakingVault](#)

Staking accepts **locked** tokens and pays yield in **liquid** ones. This is the single most important bridge in the whole design, and it deserves a sentence of its own:

Stake what you cannot sell → earn something you can.

A free player who never spends a currency can still, slowly, earn real value.

```
function stake(uint256 amount) external nonReentrant;    // locked-first, one-season lock
function unstake(uint256 amount) external nonReentrant; // at a season boundary, back to LOCKED
function claimYield() external nonReentrant;            // pays LIQUID from the toll pool
function fund(uint256 amount) external onlyRole(SINK_ROLE);
```

The yield pool is finite — it is fed only by the 20% share of real spending. Yield therefore rises when the game is busy and falls when it is quiet, which is the correct behaviour and requires no intervention. There is no fixed APR to defend, and nothing to mint if the pool runs dry.

Contract seven — [Governor](#)

Standard OpenZeppelin Governor with a snapshot token and a timelock. Voting weight comes from **liquid** balance and staked position only; locked balance carries no vote. Every parameter mentioned in this document — toll rate, burn split, tier prices, deed cap, defender advantage — is governable. The supply cap and the mint restriction are not.

TIMELOCK IS MANDATORY

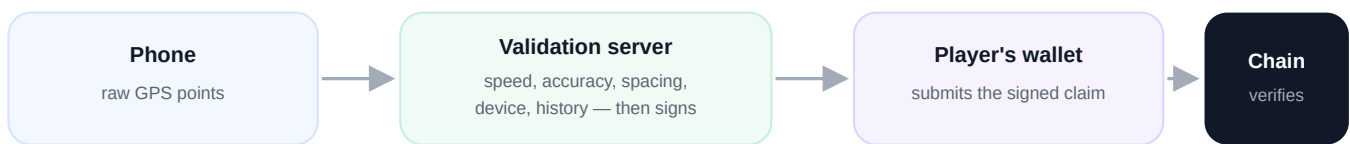
Minimum 48 hours between a passed vote and its execution. It is the only thing that lets an ordinary player see a bad parameter change coming and act before it lands.

The oracle and its signatures

A blockchain cannot see a person walking. Something has to tell it, and that something has to be impossible to lie to cheaply. This section is the exact shape of that bridge.

The problem, stated plainly

The contract knows how to pay a runner for 4.2 kilometres. It has no way of knowing whether those 4.2 kilometres happened. GPS data lives on a phone; the chain sees only what somebody submits to it. So the server validates the movement and then **signs** a small statement about it. The contract does not trust the player — it verifies a signature from a key it already knows.



Raw location never reaches the chain. Only the signed conclusion does.

The payload

```

struct SessionProof {
    address    runner;
    uint256    gross;           // tokens earned, 18 decimals
    uint32     day;             // UTC day index – anchors the daily cap
    uint256[]  crossings;       // owned cells traversed, max 40
    bytes32    routeHash;       // SHA-256 over the validated route
    uint256    nonce;
    uint64     deadline;
    bytes      signature;
}
  
```

```

// What the oracle signs, and what the contract re-derives.
bytes32 digest = keccak256(abi.encodePacked(
    block.chainid,           // ← a signature for testnet is invalid on mainnet
    address(this),           // ← a signature for one contract is invalid for another
    p.runner,
    p.gross,
    p.day,
    keccak256(abi.encodePacked(p.crossings)),
    p.routeHash,
    p.nonce,
    p.deadline
));

address signer = digest.toEthSignedMessageHash().recover(p.signature);
require(hasRole(ORACLE_ROLE, signer), "BAD_SIGNER");
require(!usedNonce[p.runner][p.nonce], "REPLAY");
require(block.timestamp <= p.deadline, "EXPIRED");
usedNonce[p.runner][p.nonce] = true;
  
```

Why each field is there

FIELD	THE ATTACK IT STOPS
-------	---------------------

<code>block.chainid</code>	Harvesting cheap signatures on a test network and replaying them on the real one. Without this, testnet is a free money faucet for mainnet.
<code>address(this)</code>	Taking a signature meant for the session contract and feeding it to the siege contract, which expects a similar shape.
<code>runner</code>	Somebody else's valid proof being submitted by your wallet.
<code>day</code>	Saving up proofs and spending them all at once to blow past the daily cap.
<code>nonce</code>	Submitting the same honest run over and over. This is the single most common exploit in this category of product.
<code>deadline</code>	A proof signed months ago under different parameters being redeemed today.

OPERATIONAL RULES FOR THE ORACLE KEY

One key, one job. The key that signs player rewards must never also be the key that runs scheduled maintenance, and neither may hold admin rights on any contract. Rotate through `ORACLE_ROLE` — granting a new signer and revoking the old one — so a compromised key is a fifteen-minute incident and not a redeployment. Keep the key in a hardware module or a managed signing service; never in an environment variable in a repository.

What the server must check before it signs

Physics

Speed ceiling around 22 m/s, monotonic timestamps, at least ten points, no more than 30% of points with poor accuracy, session under 24 hours. A car does not walk.

Continuity

No teleports between points, no session starting where the last one did not end, no impossible altitude jumps.

Identity

One account per device attestation. This is what makes running a hundred fake accounts expensive rather than free.

Privacy

Raw points are validated, hashed, and deleted on a fixed schedule. The chain stores a hash; nobody's movement history sits in a public ledger.

Roles, security, testing, migration

The specification is only half the job. This is how it gets built safely, what has to be proven before launch, and how the existing deployment becomes this one.

Who is allowed to do what

ROLE	HELD BY	MAY DO
<code>DEFAULT_ADMIN</code>	Timelock (behind Governor)	Grant and revoke every other role. Never an individual wallet.
<code>MINTER_ROLE</code>	<code>SeasonController</code> — and nothing else, ever	Create tokens, within the season schedule only.
<code>SPENDER_ROLE</code>	<code>BurnRouter</code> , <code>StakingVault</code> , <code>SiegeManager</code>	Move tokens out of a balance for one specific game action.
<code>SINK_ROLE</code>	Every contract that charges for something	Call <code>BurnRouter.route</code> .
<code>ORACLE_ROLE</code>	The signing key, alone	Sign proofs. No admin rights; cannot move a token.
<code>SIEGE_ROLE</code>	<code>SiegeManager</code>	Escrow and release a deed.
<code>PAUSER_ROLE</code>	A 2-of-3 multisig	Pause <i>claims and sieges only</i> — never withdrawals, unstaking or transfers.

THE PAUSE RULE, BECAUSE IT GETS THIS WRONG EVERYWHERE

A pause switch that can freeze people's ability to withdraw their own tokens is indistinguishable from a rug pull, and will be read as one. **Pause the taps, never the exits.** Test that a paused system still lets a user unstake and transfer.

Patterns to apply without discussion

Checks · Effects · Interactions

Validate, then write state, then call out. Every external call in `SiegeManager` and `StakingVault` comes last, under `nonReentrant` .

Pull, never push

Tolls and yield accumulate to a claimable balance. Never loop over recipients sending tokens — one reverting recipient must not block everybody else.

Bound every loop

`crossings` is capped at 40 in the struct and re-checked in the function. An unbounded array here is a gas bomb waiting for the busiest city.

No `tx.origin` , no `block.timestamp` for randomness

Authorisation uses `msg.sender` and roles. Anything that needs randomness uses a commit-reveal or an external beacon.

Custom errors, not strings

Cheaper, and they carry the offending values — which matters when a ticket says "it just failed".

Immutable where possible

Addresses wired at deployment should be `immutable` . A settable address is an unaudited upgrade path.

Upgradeability

Two contracts hold things people own: `MoveToken` and `DeedRegistry`. These should be **non-upgradeable**. Everything else — season rules, siege scoring, burn splits, staking — sits behind the standard proxy pattern with the timelock as the only admin, so game balance can change without ever putting balances or deeds at risk.

A USEFUL WAY TO THINK ABOUT IT

Ownership is permanent, rules are adjustable. If a change could alter what somebody owns, it must not be possible. If a change only alters what somebody can do next season, it should be easy.

What has to pass before anything is deployed

TEST CLASS	WHAT IT MUST PROVE
Supply invariant	After any random sequence of mints, spends, burns, stakes and sieges: <code>totalMinted - totalBurned == totalLiquid + totalLocked</code> and <code>totalMinted <= MAX_SUPPLY</code> . Run it as a fuzz test with thousands of sequences, not as three fixed cases.
Locked cannot escape	No path — transfer, approve-and-pull, staking, siege, sponsor prize, tier upgrade — turns a locked token into a liquid one except the two explicitly permitted ones.
Siege liveness	A defender who goes silent loses. A defender who transfers, sells, or lists the deed mid-siege cannot, because it is in escrow. Settlement is callable by a stranger.
Toll ceiling	A route crossing 40 owned tier-5 cells still pays no more than 5% in total. Assert the exact number, not just "less than gross".
Signature hardening	Replayed nonce reverts. Wrong chain id reverts. Wrong contract address reverts. Expired deadline reverts. Signature from a revoked oracle reverts.
Cap enforcement	Daily cap, season pool, and per-account deed cap all hold under concurrent submissions in the same block.
Pause safety	With the system paused, a user can still unstake and transfer.
Economic simulation	A multi-season run with realistic player mixes showing burn exceeding emission at the modelled participation rates — and showing where it stops being true.

Lessons carried over from the current build

The existing deployment taught us several things the hard way. These are written down so the new contracts do not repeat them.

WHAT WENT WRONG	WHAT THE NEW BUILD DOES
Constants declared in the contract but the real value computed off-chain, so the on-chain number was inert and misleading	Anything the contract does not use is deleted. If a value matters, the contract enforces it.
Reward maths re-derived on the server from the wrong base, drifting away from the contract's own answer	The server asks the contract (<code>currentRate()</code> , <code>poolFor()</code>) rather than recomputing. One source of truth.
One private key used for both signing rewards and running maintenance	Separate keys, separate roles, neither with admin rights.
Documentation describing emission one way and the contract implementing another	The schedule in §24 is generated from the contract's own <code>poolFor</code> . Where the two disagree, the contract is right and the document is a bug.
Raw location payloads with no stated retention window	A fixed TTL, enforced by a job, documented publicly, before the first real user.
Metadata pointing at a domain nobody controls any more	Metadata resolves through a contract-level base URI that governance can move.

Migration path

1. **Deploy fresh on testnet.** The new token is a new contract; the dual-state balance cannot be retrofitted onto a deployed ERC-20.
2. **Snapshot.** Take balances and zone ownership from the existing deployment at a published block.
3. **Credit as locked.** Existing balances arrive in the new token as *locked*, matching how they were earned. Existing zones become deeds at tier 1 with their original claim date.
4. **Run both for one season.** The old contracts stay readable but stop accepting new claims.
5. **Audit, then mainnet.** No mainnet deployment without a completed third-party audit of `MoveToken` , `SiegeManager` and `BurnRouter` at minimum — those three hold the money, the conflict, and the destruction.

Every term, in one line

If a word in this document was unfamiliar, it is defined here. Nothing in the glossary is new — it is a place to look things up.

Base

The blockchain MovenRun runs on. Cheap and fast enough that a game can afford to write to it.

Burn

Permanently destroying tokens. They are gone; nobody holds them afterwards.

BurnRouter

The single contract every in-game payment passes through, so all destruction is auditable in one place.

Cell

One hexagon on the map, about 0.74 km². The unit of land.

Deed

The NFT proving you minted a cell. Its id is the cell id.

Effort curve

The rule that pays a lot for the first kilometre and progressively less after that.

Escrow

Holding an asset in a contract so neither party can move it until a condition is met.

Gas

The small fee paid to the network for writing something to the chain.

Integrity

A deed's condition, from 100% down. Traffic wears it; repairing costs tokens.

Liquid

A token state. Transferable and sellable — an ordinary token.

Locked

A token state. Spendable inside the game, never transferable or sellable.

Loop capture

Running a closed path and claiming everything inside it.

Mint (deed)

Paying to convert land you hold into an NFT you own. Your earnings there become liquid from that day.

Mint (token)

Creating new tokens. Only `SeasonController` can do it, only within the schedule.

Oracle

The signing key that tells the chain a run really happened.

Season

90 days. Each one releases a smaller pool than the last.

Siege

The seven-day contest for someone else's deed.

Sink

Anything that removes tokens from circulation.

Source

Anything that creates them. There is exactly one.

Stake

Committing tokens for a period in exchange for a share of a pool.

Supply cap

1 billion. Enforced in code, not policy.

Tier

A deed's level, 1 to 5. Multiplies what crossings pay you; adds no per-day drip.

Timelock

The mandatory delay between a vote passing and taking effect.

Toll

2% of a runner's session reward, paid to the owners of cells they crossed. Capped at 5% in total.

Wallet

The app that holds your tokens and signs on your behalf.

THREE SENTENCES THAT HOLD THE WHOLE DESIGN

One: newly minted tokens are locked, tokens that already existed move freely. **Two:** spending always consumes locked tokens first, so the players who never pay are the ones doing most of the burning. **Three:** land earns from traffic, not from time, so no amount of capital can turn ownership into a machine that prints money on its own.

The whole thing on one page

If somebody reads nothing else, this is what they need.

1B

Hard supply cap, enforced in the token contract. There is no path to a 1,000,000,001st token.

−10%

Every season's reward pool against the last, down to a permanent floor. Roughly 43 years of runway.

70%

Of every token spent in the game is destroyed. The other 30% goes to stakers and back into future seasons.

The game

You walk or run a closed loop. Everything inside it becomes yours on the map. Other people can challenge you for it over seven days, and if you never show up, you lose it — including to a single determined attacker. When somebody crosses ground you own, 2% of what they earned goes to you, and no route ever pays more than 5% in total no matter how many owners it crosses. Upgrading land multiplies that income, but only where people actually walk.

The token

There is one token with two states. Everything you earn arrives locked: fully usable inside the game, impossible to sell. If you want your earnings to become real, sellable value, you mint your land — from that day forward, what that land earns arrives liquid. Or you stake what you have, and the yield comes back liquid. Free players are never excluded and never stuck; they simply take the longer road.

Why it does not collapse

The source is fixed

A season pool set in code, shrinking on schedule. More players do not mean more tokens — they mean the same pool shared further, which is how a real economy behaves.

The sinks are elastic

Every sink scales with activity: more players means more sieges, more crossings, more wear, more repair. Growth increases destruction faster than it increases creation.

Free players burn

Because spending consumes locked tokens first, a large free population is not a cost. It is the deflationary engine. The more of them there are, the harder the supply falls.

The business is separate

Sponsors, subscriptions and events are ordinary revenue in ordinary money. If the token were worth nothing tomorrow, the company would still have customers.

THE TEST EVERY PROPOSAL HAS TO PASS

Before any new feature ships, one question: **does spending here create a new earner?** If a sink hands somebody else a new way to mint tokens, it is not a sink — it is a redistribution with extra steps, and it is the mechanism behind every failure in this category. Ours are checked against that question one by one in §14 and §18.